

Практическое задание (исправленный отчёт)

Задача линейного программирования

Симплекс-метод · Вариант 7 · канон = min W

Соколов Никита Константинович

Группа К3339 · Поток 1.2

Состав отчёта

1–3. Рукописное решение ($W = -z \rightarrow \min$, двухэтапный симплекс)

4. Проверка в Excel (Поиск решения / Solver)

5. Программная реализация (автоприведение к канону + min)

6. Вывод программы

Ответ: $x^* = (6, 0, 0, 9)$, $W^* = -54$, $z^* = 54$

1. Рукописное решение (стр. 1) — канон $\min W = -z$

Семья Купцова Климатический

Уровень КБЗЗ9, К-ПРК 1.2

7. Вариант

3/11, шестых марта

1. Исходная задача

максимизировать $z = 3x_1 + 2x_2 + x_3 + 4x_4$

при условиях: $2x_1 + x_2 + x_3 \leq 12$

$$x_2 + x_3 + x_4 = 9$$

$$x_1 + x_4 = 4$$

$$x_1, x_2, x_3, x_4 \geq 0$$

2. Приведение к каноническому виду

$$W = -z = -3x_1 - 2x_2 - x_3 - 4x_4 - \min$$

от переменных.

$$x_5 \geq 0 \text{ (для } \leq \text{)}, x_6 \geq 0 \text{ (для } = \text{)}$$

$$2x_1 + x_2 + x_3 + x_5 = 12$$

$$x_2 + x_3 + x_4 = 9$$

$$x_1 + x_4 - x_6 = 4$$

$$x_1, \dots, x_6 \geq 0$$

3. Крайняя задача

исключительно $x_1, x_8 \geq 0$

$$2x_1 + x_2 + x_3 + x_5 = 12$$

$$x_2 + x_3 + x_4 + x_7 = 9$$

$$x_1 + x_4 - x_6 + x_8 = 4$$

$$\varphi = x_1 + x_8 - \min$$

Базис: $x_5 = 12, x_7 = 9, x_8 = 4$

$$x_7 = 9 - x_2 - x_3 - x_4$$

$$x_8 = 4 - x_1 - x_4 + x_6$$

$$\varphi = 13 - x_1 - x_2 - x_3 - 2x_4 + x_6 - \min$$

2. Рукописное решение (стр. 2) — вспомогательная + переход

Таблица 1

Базис	x_1	x_2	x_3	x_4	x_5	b
x_5	2	1	1	0	0	7
x_2	0	1	1	1	0	5
x_4	1	0	0	1	-1	4
φ	-1	-1	-1	-2	1	-13

Вводим $x_4(-2)$, отбрасываем $9/1=9$, $4/1=4$ - вводим x_4

Таблица 2

Базис	x_1	x_2	x_3	x_4	x_5	b
x_5	2	1	1	0	0	7
x_2	-1	1	1	0	1	5
x_4	1	0	0	1	-1	4
φ	1	-1	-1	0	-1	-5

Вводим $x_2(-1)$

Отбрасываем: $5/1=5$ - вводим x_2

Таблица 3

Базис	x_1	x_2	x_3	x_4	x_5	b
x_5	3	0	0	0	-1	7
x_2	-1	1	1	0	1	5
x_4	1	0	0	1	-1	4
φ	0	0	0	0	0	0

$\varphi^* = 0$ - оптимальный базис

Искусственные $x_3 = x_5 = 0$ исключены

Базис: $x_5 = 7$, $x_2 = 5$, $x_4 = 4$

Небазисные: x_1, x_3, x_6

4. Переход к основной задаче

$$x_5 = 7 - 3x_1 + x_6$$

$$x_2 = 5 + x_1 - x_3 - x_6$$

$$x_4 = 4 - x_1 + x_6$$

$$W = -3x_1 - 2x_2 - x_3 - 4x_4 = -26 - x_1 + x_3 - 2x_6 - \text{min}$$

Таблица 4:

Базис	x_1	x_3	x_6	b
x_5	3	0	-1	7
x_2	-1	1	1	5
x_4	1	0	-1	4
W	-1	1	-2	-26

Вводим $x_6(-2)$

Отбрасываем: $5/1=5$ - вводим x_2

3. Рукописное решение (стр. 3) — основная задача + ответ

$$\begin{aligned}x_1 &= 84 - x_3 - x_2 \\x_5 &= 12 - 2x_1 - x_3 - x_2 \\x_4 &= 9 - x_3 - x_2 \\W &= -36 - 3x_1 + 3x_3 + 2x_2\end{aligned}$$

Таблица 5:

Базис	x_1	x_3	x_2	b
x_5	2	1	1	12
x_6	-1	1	1	5
x_4	0	1	1	9
W	-3	3	2	-36

Вводим x_2 (-2)

отщипываем: $12/2 = 6$ — отщипываем x_5

$$\begin{aligned}x_1 &= 6 - \frac{1}{2}x_5 - \frac{1}{2}x_3 - \frac{1}{2}x_2 \\x_6 &= 11 - \frac{1}{2}x_5 - \frac{1}{2}x_3 - \frac{1}{2}x_2 \\x_4 &= 9 - x_3 - x_2 \\W &= -54 + \frac{3}{2}x_5 + \frac{9}{2}x_3 + \frac{7}{2}x_2\end{aligned}$$

Таблица 6:

Базис	x_5	x_3	x_2	b
x_1	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	6
x_6	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	11
x_4	0	1	1	9
W	$\frac{3}{2}$	$\frac{9}{2}$	$\frac{7}{2}$	-54

Все элементы в строке $W \geq 0$,
можно остановиться для min

5. Ответ: оптимальная точка $x^* = (x_1, x_2, x_3, x_4) = (6, 0, 0, 9)$
 $W^* = -54$
 $z^* = -W^* = 54$ (Знач. функции критерия)

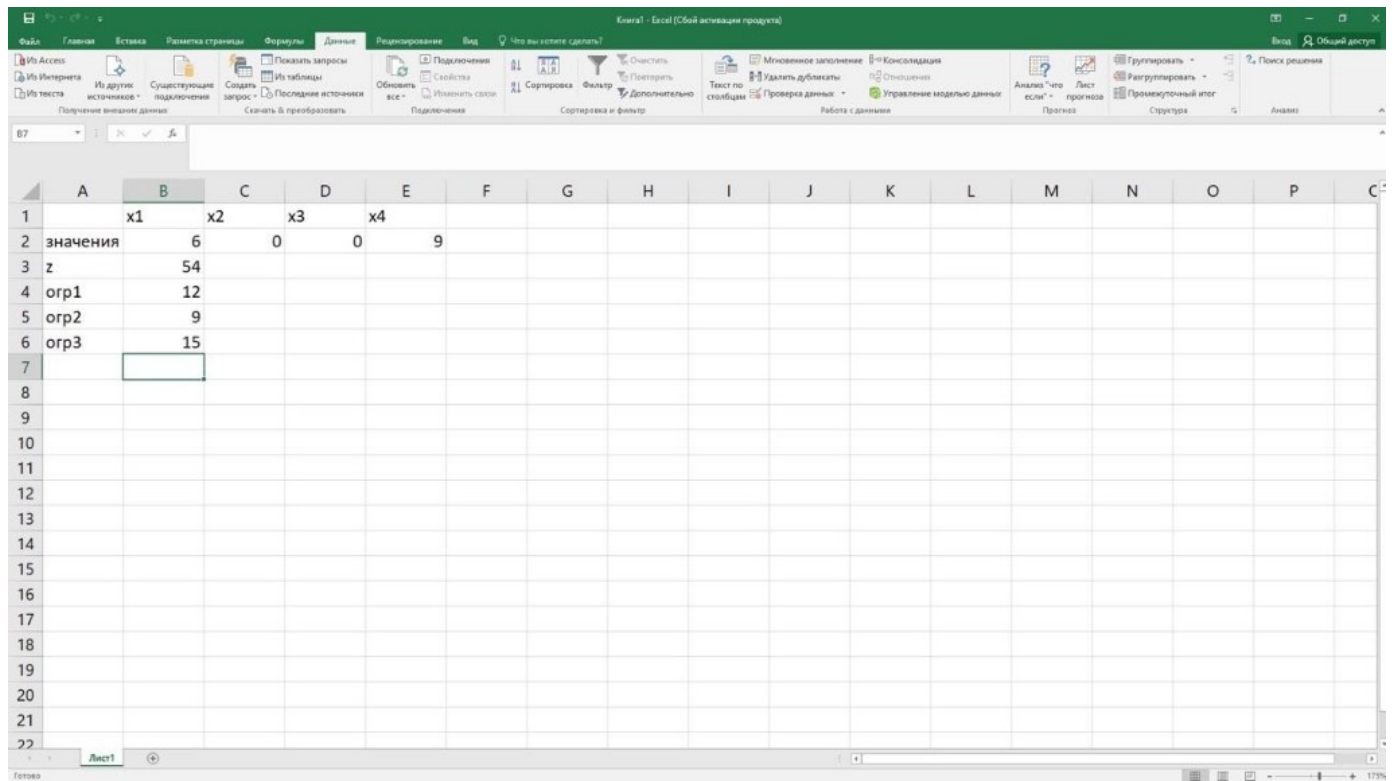
проверка: $2 \cdot 6 + 0 + 0 = 12 \leq 12$
 $0 + 0 + 9 = 9$
 $6 + 9 = 15 \leq 14$

6. Базис: $x_1 = 6$; $x_2 = 0$; $x_3 = 0$; $x_4 = 9$; $z = 54$; $W = -54$

7. Проверка условий:

Для меня было затруднительно перейти к канонической форме симплекс-метода. Сейчас было важно аккуратно заполнить таблицу и не ошибиться в значениях b_i и значений переменных, но по формулам b_i и W проверю и подставлю в ограничения

4. Проверка в Excel (Solver)



5. Программная реализация (Python)

Файл `simplex_v7.py`: программа сама приводит задачу к канону ($\min W = -z$, дополнительные/избыточные/искусственные переменные) и решает двухэтапным симплексом.

```
from __future__ import annotations

from dataclasses import dataclass, field

Number = float
EPS = 1e-10

def fmt(x: Number) -> str:
    if abs(x) < EPS:
        return "0"
    if abs(x - round(x)) < EPS:
        return str(int(round(x)))
    return f"{x:.4g}"

class Expr:
    def __init__(self, const: Number = 0.0, coeffs: dict[str, Number] | None = None):
        self.const = float(const)
        self.coeffs = {k: float(v) for k, v in (coeffs or {}).items() if abs(v) > EPS}

    def __add__(self, other: "Expr | Number") -> "Expr":
        if not isinstance(other, Expr):
            return Expr(self.const + float(other), self.coeffs)
        coeffs = dict(self.coeffs)
        for k, v in other.coeffs.items():
            coeffs[k] = coeffs.get(k, 0.0) + v
        return Expr(self.const + other.const, coeffs)

    def __sub__(self, other: "Expr | Number") -> "Expr":
        if not isinstance(other, Expr):
            return Expr(self.const - float(other), self.coeffs)
        return self + Expr(-other.const, {k: -v for k, v in other.coeffs.items()})

    def __mul__(self, k: Number) -> "Expr":
        k = float(k)
        return Expr(self.const * k, {n: v * k for n, v in self.coeffs.items()})

    __rmul__ = __mul__

    def __neg__(self) -> "Expr":
        return self * (-1)

    def substitute(self, var: str, replacement: "Expr") -> "Expr":
        if var not in self.coeffs:
            return Expr(self.const, self.coeffs)
        a = self.coeffs[var]
        rest = Expr(self.const, {k: v for k, v in self.coeffs.items() if k != var})
        return rest + replacement * a

    def __str__(self) -> str:
        parts: list[str] = []
        if abs(self.const) > EPS or self.coeffs:
            parts.append(fmt(self.const))
        for name in sorted(self.coeffs):
            v = self.coeffs[name]
            if abs(v) < EPS:
                continue
            sign = "+" if v > 0 else "-"
            av = abs(v)
            term = name if abs(av - 1) < EPS else f"{fmt(av)}*{name}"
            if not parts:
                parts.append(term if v > 0 else f"-{term}")
            else:
                parts.append(f"{sign} {term}")
        return " ".join(parts) if parts else "0"

@dataclass
class Constraint:
    coeffs: dict[str, Number]
```

5. Программная реализация (продолжение 2/6)

```
sense: str
rhs: Number

@dataclass
class LPPProblem:
    sense: str
    objective: dict[str, Number]
    constraints: list[Constraint]
    variables: list[str]

@dataclass
class CanonicalLP:
    W_coeffs: dict[str, Number]
    equalities: list[dict[str, Number]]
    variables: list[str]
    artificial: list[str] = field(default_factory=list)
    row_basic: list[str] = field(default_factory=list)
    notes: list[str] = field(default_factory=list)

def to_canonical(problem: LPPProblem) -> CanonicalLP:
    notes: list[str] = []
    vars_ = list(problem.variables)
    next_id = len(vars_) + 1

    def new_var(prefix: str = "x") -> str:
        nonlocal next_id
        name = f"{prefix}{next_id}"
        next_id += 1
        vars_.append(name)
        return name

    if problem.sense == "max":
        W = {v: -c for v, c in problem.objective.items()}
        notes.append("Исходная задача на max z → канон: min W = -z")
        notes.append(
            "W = "
            + " + ".join(f"({fmt(c)}) * {v}" for v, c in W.items())
            + " → min"
        )
    elif problem.sense == "min":
        W = dict(problem.objective)
        notes.append("Исходная задача уже на min → W совпадает с критерием")
    else:
        raise ValueError("sense должен быть 'max' или 'min'")

    equalities: list[dict[str, Number]] = []
    row_basic: list[str | None] = []

    for i, cons in enumerate(problem.constraints, start=1):
        row = {v: float(cons.coefs.get(v, 0.0)) for v in problem.variables}
        b = float(cons.rhs)
        sense = cons.sense
        basic: str | None = None

        if sense == "<=":
            s = new_var("x")
            row[s] = 1.0
            basic = s
            notes.append(f"Огр.{i} (<=): введена дополнительная {s} >= 0 (базис)")
        elif sense == ">=":
            s = new_var("x")
            row[s] = -1.0
            notes.append(f"Огр.{i} (>=): введена избыточная {s} >= 0")
        elif sense == "=":
            notes.append(f"Огр.{i} (=): равенство")
        else:
            raise ValueError(f"Неизвестный тип ограничения: {sense}")

        if b < -EPS:
```

5. Программная реализация (продолжение 3/6)

```
        row = {k: -v for k, v in row.items()}
        b = -b
        notes.append(f"Орп.{i}: умножили на -1, чтобы b >= 0")

    row["__b__"] = b
    equalities.append(row)
    row_basic.append(basic)

artificial: list[str] = []
for i, basic in enumerate(row_basic):
    if basic is None:
        a = new_var("x")
        equalities[i][a] = 1.0
        row_basic[i] = a
        artificial.append(a)
        notes.append(f"Орп.{i + 1}: введена искусственная {a} >= 0 (базис)")

W_full = {v: float(W.get(v, 0.0)) for v in vars_}

return CanonicalLP(
    W_coefs=W_full,
    equalities=equalities,
    variables=[v for v in vars_ if v not in artificial],
    artificial=artificial,
    row_basic=[b for b in row_basic if b is not None],
    notes=notes,
)

def print_tableau(
    title: str,
    basics: dict[str, Expr],
    objective: Expr,
    nonbasic: list[str],
    obj_name: str,
    negate_obj_rhs: bool = False,
) -> None:
    print(f"\n{title}")
    header = f"'Базис':>6} |" + "".join(f"{n:>8}" for n in nonbasic) + f" | {'b':>8}"
    print(header)
    print("-" * len(header))
    for bname, expr in basics.items():
        row = f"{bname:>6} |"
        for n in nonbasic:
            row += f"{fmt(-expr.coefs.get(n, 0.0)):>8}"
        row += f" | {fmt(expr.const):>8}"
        print(row)
    row = f"{obj_name:>6} |"
    for n in nonbasic:
        row += f"{fmt(objective.coefs.get(n, 0.0)):>8}"
    rhs = -objective.const if negate_obj_rhs else objective.const
    row += f" | {fmt(rhs):>8}"
    print(row)

def pivot_dictionary(
    basics: dict[str, Expr],
    objective: Expr,
    nonbasic: list[str],
    enter: str,
    leave: str,
) -> tuple[dict[str, Expr], Expr, list[str]]:
    leave_expr = basics[leave]
    a = leave_expr.coefs.get(enter, 0.0)
    if abs(a) < EPS:
        raise RuntimeError(f"Нулевой разрешающий элемент для {enter}")

    enter_expr = Expr(-leave_expr.const / a, {leave: 1.0 / a})
    for n, v in leave_expr.coefs.items():
        if n == enter:
            continue
        enter_expr.coefs[n] = enter_expr.coefs.get(n, 0.0) - v / a
```

5. Программная реализация (продолжение 4/6)

```
new_basics: dict[str, Expr] = {}
for bname, expr in basics.items():
    if bname == leave:
        continue
    new_basics[bname] = expr.substitute(enter, enter_expr)
new_basics[enter] = enter_expr

new_obj = objective.substitute(enter, enter_expr)
new_nb = [leave if n == enter else n for n in nonbasic]
for bname in list(new_basics):
    new_basics[bname] = Expr(new_basics[bname].const, new_basics[bname].coeffs)
new_obj = Expr(new_obj.const, new_obj.coeffs)
return new_basics, new_obj, new_nb

def solve_min(
    basics: dict[str, Expr],
    objective: Expr,
    nonbasic: list[str],
    title: str,
    obj_name: str,
    negate_obj_rhs: bool = False,
) -> tuple[dict[str, Expr], Expr, list[str]]:
    step = 0
    print_tableau(f"{title}, шаг {step}", basics, objective, nonbasic, obj_name, negate_obj_rhs)
    while True:
        enter = None
        best = -EPS
        for n in nonbasic:
            coef = objective.coeffs.get(n, 0.0)
            if coef < best:
                best = coef
                enter = n
        if enter is None:
            break

        ratios: list[tuple[Number, str]] = []
        for bname, expr in basics.items():
            a_expr = expr.coeffs.get(enter, 0.0)
            if a_expr < -EPS:
                ratios.append((expr.const / (-a_expr), bname))
        if not ratios:
            raise RuntimeError(f"{title}: задача неограничена")
        leave = min(ratios)[1]

        print(f"\nШаг {step + 1}: вводим {enter}, выводим {leave}")
        basics, objective, nonbasic = pivot_dictionary(basics, objective, nonbasic, enter, leave)
        step += 1
        print_tableau(f"{title}, после шага {step}", basics, objective, nonbasic, obj_name, negate_obj_rhs)
    return basics, objective, nonbasic

def build_phase1(canon: CanonicalLP) -> tuple[dict[str, Expr], Expr, list[str]]:
    all_vars: list[str] = []
    for row in canon.equalities:
        for k in row:
            if k != "__b__" and k not in all_vars:
                all_vars.append(k)

    basics: dict[str, Expr] = {}
    for row, basic in zip(canon.equalities, canon.row_basic):
        b = row["__b__"]
        coeffs = {}
        for v, coef in row.items():
            if v == "__b__" or v == basic:
                continue
            coeffs[v] = -coef
        basics[basic] = Expr(b, coeffs)

    nonbasic = [v for v in all_vars if v not in basics]
```

5. Программная реализация (продолжение 5/6)

```
phi = Expr(0)
for a in canon.artificial:
    if a in basics:
        phi = phi + basics[a]
    else:
        phi = phi + Expr(0, {a: 1.0})

return basics, phi, nonbasic

def main() -> None:
    print("=" * 68)
    print("Вариант 7. Двухэтапный симплекс (канон = задача на min)")
    print("=" * 68)

    problem = LPProblem(
        sense="max",
        objective={"x1": 3, "x2": 2, "x3": 1, "x4": 4},
        variables=["x1", "x2", "x3", "x4"],
        constraints=[
            Constraint({"x1": 2, "x2": 1, "x3": 1}, "<=", 12),
            Constraint({"x2": 1, "x3": 1, "x4": 1}, "=", 9),
            Constraint({"x1": 1, "x4": 1}, ">=", 4),
        ],
    )

    print("\n0. Исходная задача")
    print("max z = 3x1 + 2x2 + x3 + 4x4")
    print("  2x1 + x2 + x3      <= 12")
    print("      x2 + x3 + x4    = 9")
    print("  x1          + x4    >= 4")
    print("  xi >= 0")

    canon = to_canonical(problem)

    print("\n1. Автоматическое приведение к каноническому виду")
    for note in canon.notes:
        print("  •", note)

    print("\nКаноническая система равенств:")
    for i, row in enumerate(canon.equalities, 1):
        parts = []
        first = True
        for v in sorted((k for k in row if k != "__b__"), key=lambda s: (int(s[1:]) if s[1:].isdigit() else 99, s)):
            c = row[v]
            if abs(c) < EPS:
                continue
            if first:
                parts.append(f"{fmt(c)}{v}" if abs(c) != 1 else (v if c > 0 else f"-{v}"))
                first = False
            else:
                sign = "+" if c > 0 else "-"
                ac = abs(c)
                parts.append(f"{sign} {fmt(ac)} {v}" if abs(ac-1)>EPS else '{v}'.replace(" ", ""))
        print(f"  ({i}) " + " ".join(parts) + f" = {fmt(row['__b__'])}")

    print("Искусственные переменные:", " ".join(canon.artificial) if canon.artificial else "нет")
    print("min W =", Expr(0, {v: c for v, c in canon.W_coeffs.items() if abs(c) > EPS}), "→ min")

    print("\n2. Вспомогательная задача: φ = сумма искусственных → min")
    basics, phi, nonbasic = build_phase1(canon)
    basics, phi, nonbasic = solve_min(
        basics, phi, nonbasic, "Фаза I (вспомогательная)", "φ", negate_obj_rhs=True
    )

    if abs(phi.const) > EPS:
        raise RuntimeError(f"Нет допустимых решений, φ* = {phi.const}")
    print(f"\nφ* = {fmt(phi.const)} → допустимый базис найден")

    art = set(canon.artificial)
    for a in list(basics):
        if a in art:
```

5. Программная реализация (продолжение 6/6)

```
        if abs(basics[a].const) > EPS:
            raise RuntimeError(f"Искусственная {a} осталась в базисе")
        basics.pop(a)
    nonbasic = [n for n in nonbasic if n not in art]
    for bname in list(basics):
        basics[bname] = Expr(
            basics[bname].const,
            {k: v for k, v in basics[bname].coeffs.items() if k not in art},
        )

    def var_expr(name: str) -> Expr:
        if name in basics:
            return basics[name]
        if name in nonbasic:
            return Expr(0, {name: 1})
        return Expr(0)

    W = Expr(0)
    for v, c in canon.W_coeffs.items():
        if abs(c) > EPS:
            W = W + c * var_expr(v)

    print(f"\n3. Основная задача (канон): W = {W} → min")
    basics, W, nonbasic = solve_min(basics, W, nonbasic, "Фаза II (основная, min W)", "W")

    values = {v: 0.0 for v in problem.variables}
    for bname, expr in basics.items():
        if bname in values:
            values[bname] = expr.const

    x1, x2, x3, x4 = values["x1"], values["x2"], values["x3"], values["x4"]
    W_star = W.const
    z_star = -W_star

    print("\n" + "=" * 68)
    print("ОТВЕТ")
    print(f"x* = ({fmt(x1)}, {fmt(x2)}, {fmt(x3)}, {fmt(x4)})")
    print(f"W* = {fmt(W_star)} (минимум канонической задачи)")
    print(f"z* = -W* = {fmt(z_star)} (значение исходного max)")
    print("Проверка ограничений:")
    print(f" 2x1+x2+x3 = {fmt(2*x1+x2+x3)} <= 12")
    print(f" x2+x3+x4 = {fmt(x2+x3+x4)} = 9")
    print(f" x1+x4 = {fmt(x1+x4)} >= 4")
    print("=" * 68)

if __name__ == "__main__":
    main()
```

6. Вывод программы

Команда: `python3 simplex_v7.py`

Результат: $x^* = (6, 0, 0, 9)$, $W^* = -54$, $z^* = 54$ — совпадает с рукописью и Excel.

```
=====
Вариант 7. Двухэтапный симплекс (канон = задача на min)
=====

0. Исходная задача
max z = 3x1 + 2x2 + x3 + 4x4
  2x1 + x2 + x3    <= 12
        x2 + x3 + x4 = 9
  x1              + x4 >= 4
  xi >= 0

1. Автоматическое приведение к каноническому виду
• Исходная задача на max z → канон: min W = -z
• W = (-3)*x1 + (-2)*x2 + (-1)*x3 + (-4)*x4 → min
• Огр.1 (<=): введена дополнительная x5 >= 0 (базис)
• Огр.2 (=): равенство
• Огр.3 (>=): введена избыточная x6 >= 0
• Огр.2: введена искусственная x7 >= 0 (базис)
• Огр.3: введена искусственная x8 >= 0 (базис)

Каноническая система равенств:
(1) 2x1 + x2 + x3 + x5 = 12
(2) x2 + x3 + x4 + x7 = 9
(3) x1 + x4 - x6 + x8 = 4
Искусственные переменные: x7, x8
min W = -3*x1 - 2*x2 - x3 - 4*x4 → min

2. Вспомогательная задача: φ = сумма искусственных → min

Фаза I (вспомогательная), старт
базис |      x1      x2      x3      x4      x6 |      b
-----|-----
x5 |      2      1      1      0      0 |     12
x7 |      0      1      1      1      0 |      9
x8 |      1      0      0      1     -1 |      4
φ  |     -1     -1     -1     -2      1 |    -13

Шаг 1: вводим x4, выводим x8

Фаза I (вспомогательная), после шага 1
базис |      x1      x2      x3      x8      x6 |      b
-----|-----
x5 |      2      1      1      0      0 |     12
x7 |     -1      1      1     -1      1 |      5
x4 |      1      0      0      1     -1 |      4
φ  |      1     -1     -1      2     -1 |     -5

Шаг 2: вводим x2, выводим x7

Фаза I (вспомогательная), после шага 2
базис |      x1      x7      x3      x8      x6 |      b
-----|-----
x5 |      3     -1      0      1     -1 |      7
x4 |      1      0      0      1     -1 |      4
x2 |     -1      1      1     -1      1 |      5
φ  |      0      1      0      1      0 |      0

φ* = 0 → допустимый базис найден
```

6. Вывод программы (продолжение 2/2)

3. Основная задача (канон): $W = -26 - x_1 + x_3 - 2x_6 \rightarrow \min$

Фаза II (основная, $\min W$), старт

базис		x_1	x_3	x_6		b

x_5		3	0	-1		7
x_4		1	0	-1		4
x_2		-1	1	1		5
W		-1	1	-2		-26

Шаг 1: вводим x_6 , выводим x_2

Фаза II (основная, $\min W$), после шага 1

базис		x_1	x_3	x_2		b

x_5		2	1	1		12
x_4		0	1	1		9
x_6		-1	1	1		5
W		-3	3	2		-36

Шаг 2: вводим x_1 , выводим x_5

Фаза II (основная, $\min W$), после шага 2

базис		x_5	x_3	x_2		b

x_4		0	1	1		9
x_6		0.5	1.5	1.5		11
x_1		0.5	0.5	0.5		6
W		1.5	4.5	3.5		-54

=====

ОТВЕТ

$x^* = (6, 0, 0, 9)$

$W^* = -54$ (минимум канонической задачи)

$z^* = -W^* = 54$ (значение исходного max)

Проверка ограничений:

$2x_1 + x_2 + x_3 = 12 \leq 12$

$x_2 + x_3 + x_4 = 9 = 9$

$x_1 + x_4 = 15 \geq 4$

=====